

Mini-Projecto 2 — RAG Pipeline

SGAIM 2025/2026 · 15% da nota final

O que vão construir

Um sistema de Retrieval-Augmented Generation (RAG) sobre um corpus à vossa escolha. O sistema recebe uma pergunta em linguagem natural, procura os documentos mais relevantes, e gera uma resposta fundamentada nesses documentos — não inventada.

No MP1 construíram o motor (o GPT). No MP2, dão-lhe **memória** — acesso a conhecimento externo que não cabe na context window e que não existia quando o modelo foi treinado.

Ponto de partida: a aula 07

Na aula 07 construíram uma pipeline RAG completa em Python puro: `load_corpus` → `chunk_fixed_size` → `embeddings` → `ChromaDB` → `rag_query`. Isso **já é** a espinha técnica do MP2. Peguem nesse código e adaptem-no ao vosso corpus.

As três adaptações operacionais estão no final do guião da aula 07 (“Ponte para o MP2”):

- Embeddings via Ollama (`nomic-embed-text`) em vez de `sentence-transformers`
- ChromaDB **persistente** (`PersistentClient`) — para não re-embedar a cada execução
- Extração de PDF com `pypdf` (só se o vosso corpus for PDF)

Estas não são as decisões do projecto — são cola operacional. As decisões vêm a seguir.

O pipeline e as decisões

Estágio	Decisões que tomam
Ingestão	Que corpus? Que formato?
Chunking	Tamanho? Estratégia (fixed, sentence, semantic)? Overlap?
Embedding	Que modelo? Dimensionalidade?
Vector Store	ChromaDB, FAISS, ou outro?
Retrieval	Quantos chunks? Filtros de metadata?
Generation	Que prompt? Que modelo? Cita fontes?

Cada uma destas decisões afecta o resultado. **O projecto não é montar o pipeline — é compreender e justificar as decisões.** A aula 07 dá-vos uma implementação default; o vosso trabalho é perceber o que muda quando mudam algum destes parâmetros.

Calendário

Semana	O que acontece	O que devem ter pronto
7	RAG introduzido; MP2 lançado	Corpus escolhido, pipeline da aula 07 a correr sobre o vosso corpus

Semana	O que acontece	O que devem ter pronto
8	RAG: avaliação e estratégias (recall@k, faithfulness, hybrid search, re-ranking)	Benchmark de 10 perguntas criado; pelo menos 1 comparação feita
9	Refinamento	2 comparações completas; relatório em progresso
10	Entrega + discussão	Tudo entregue; discussão breve na aula

Espera-se trabalho **fora das aulas**. As aulas das semanas 7–8 cobrem a teoria que o projecto implementa — usem-nas para esclarecer dúvidas.

O corpus

A escolha do corpus **condiciona toda a qualidade do projecto**.

Critérios:

- **Fora dos pesos do LLM.** Regulamentos internos, documentação privada, ou material suficientemente específico. Wikipedia, docs da OpenAI/HuggingFace, livros clássicos ou letras de música populares **não servem** — estão nos pesos.
- **Verificável por vocês.** Devem conhecer o conteúdo bem o suficiente para julgar se a resposta está certa, errada ou parcial.
- **Pelo menos 10 documentos** ou ~20 páginas de texto. Abaixo disto, o retrieval não tem trabalho para fazer.

Teste rápido: peguem em 3 perguntas do vosso corpus e colem-nas no ChatGPT/Claude **sem colar o corpus**. Se responderem bem às três, o corpus está nos pesos — escolham outro. Se disserem “não sei” ou errarem, o vosso RAG tem trabalho real para fazer.

Formato: comecem com markdown ou texto simples. PDFs absorvem horas de limpeza que tiram tempo à análise — e a análise vale metade da nota.

O que entregam

1. Código

O pipeline completo, que corre. Framework à escolha (Python puro, LangChain, LlamaIndex, outro). Deve:

- Aceitar uma pergunta em texto
- Recuperar chunks relevantes do vector store
- Gerar uma resposta usando um LLM com os chunks como contexto
- Funcionar com um modelo local (Ollama)

A escolha de framework não afecta a nota — o que importa é saber explicar o que o framework faz por vocês.

2. Relatório técnico (PDF, máximo 10 páginas)

Um relatório com as seis secções abaixo. O limite de 10 páginas inclui tabelas e figuras. Prioriza precisão sobre volume — *duas frases precisas valem mais do que um parágrafo vago*.

2.1 Corpus e escolha (~1 página) Que corpus escolheram e porquê. Como testaram que está **fora dos pesos** do LLM (ver “Teste rápido” na secção O corpus). Que alternativas consideraram e rejeitaram.

2.2 Pipeline e decisões (~2 páginas) Para cada estágio — ingestão, chunking, embedding, vector store, retrieval, generation — registar a decisão tomada, as alternativas consideradas, e o porquê. Se usaram framework, o que é que ele faz que não poderiam fazer à mão em 10 linhas?

2.3 Benchmark: 10 perguntas (~1,5 páginas) Tabela com a avaliação manual de cada resposta:

Campo	O que preencher
Pergunta	A pergunta que fizeram ao sistema
Resposta esperada	O que a resposta correcta deveria ser
Chunks relevantes?	Sim/Parcial/Não
Resposta do sistema	O que o sistema respondeu
Correcta?	Sim/Parcial/Não
Fiel ao contexto?	Sim/Não (baseou-se nos chunks ou inventou?)
Notas	O que falhou, o que surpreendeu

Incluam **perguntas fáceis, médias e armadilhas** — as armadilhas são perguntas cuja resposta **não existe** no corpus. Se o sistema responde com confiança a algo que não lá está, está a alucinar. Esse é o modo de falha mais importante de documentar.

Sugestão: 5 fáceis + 3 médias + 2 armadilhas. Outra distribuição é aceitável desde que a justifiquem.

2.4 Comparação 1 — chunking (~1,5–2 páginas) Comparação central da aula 07: pelo menos duas estratégias de chunking (ex. fixed 500 vs fixed 200, ou fixed vs sentence-level), medidas sobre o mesmo benchmark. Protocolo **predict → measure → explain**:

- **Predict:** antes de correr, escrever o que esperam que aconteça
- **Measure:** correr a variante, medir sobre o benchmark
- **Explain:** porque aconteceu o que aconteceu?

A previsão, feita antes de correr, faz parte da avaliação. Medir contra o mesmo benchmark nos dois cenários (base-line vs variante) é obrigatório — “ficou melhor à vista” não conta.

2.5 Comparação 2 — à escolha (~1,5–2 páginas) Mesma estrutura (predict → measure → explain), noutro eixo.

Recomendação forte: implementem uma **técnica avançada de retrieval** (semana 8) em vez de variar apenas hiperparâmetros. O ganho de aprendizagem é muito maior, e a análise fica mais defensável na discussão.

Opções recomendadas (técnicas avançadas — semana 8):

- **Reranking com cross-encoder** — padrão *retrieve 50, rerank to 5*. Costuma dar o maior ganho por linha de código.
- **Hybrid search** (BM25 + dense + RRF) — cobre queries com termos técnicos, códigos, acrónimos.
- **Query rewriting** (multi-query expansion) — expande cobertura lexical quando a pergunta do user usa vocabulário diferente do corpus.
- **HyDE** (Hypothetical Document Embeddings) — alinha o espaço da query com o espaço dos documentos; útil em domínios técnicos.

Opções alternativas (hiperparâmetros — ganhos mais marginais):

- Modelo de embedding (ex: `nomic-embed-text` vs `all-minilm`)
- Número de chunks recuperados (top-3 vs top-5)
- Prompt de geração (“responde com base no contexto” vs “cita as fontes”)
- Modelo de geração (ex: `llama3.2:3b` vs `qwen2.5:7b`)
- Ollama (local) vs API (OpenAI/Anthropic)

2.6 Failure analysis + o que mudariam (~1 página) Onde o sistema falha e porquê. As armadilhas — admite ignorância ou inventa? Qual é a falha mais interessante que encontraram? Se o modelo inventa uma resposta, o problema é do retrieval ou da geração? O que mudariam com mais tempo?

3. Discussão breve (na aula, semana 10)

~5 minutos por grupo. O docente pode perguntar:

- “Porque escolheste chunks de 500 e não de 200?”
- “Mostra-me uma pergunta onde o sistema falha. Porquê?”

- “Se o modelo inventa uma resposta, o problema é do retrieval ou da geração?”
- “O que mudarias se refizesse o projecto?”

Não é interrogatório — é verificação de que compreendem as decisões que tomaram.

Critérios de avaliação

Critério	Peso	O que significa
Pipeline funcional	25%	Corre, aceita perguntas, recupera chunks, gera respostas.
Compreensão	50%	Decisões justificadas. Failure analysis honesta. Discussão confirma compreensão genuína.
Avaliação	25%	Benchmark bem desenhado (inclui fáceis, médias e armadilhas). Comparações com método (predict → measure → explain).

Compreensão vale **metade da nota**. Um pipeline sofisticado que não conseguem explicar vale menos do que um pipeline simples com análise excelente.

Regras

Trabalho em grupo

- Grupos de **1 ou 2** alunos (os mesmos do MP1, ou diferentes).
- Se trabalharem a par: ambos devem compreender cada decisão. A discussão será dirigida a cada um individualmente.

Uso de assistentes de IA

Devem usar assistentes de IA — mas com um protocolo específico:

1. **Use o prompt de design critic** (ai-design-critic-prompt.md no Moodle, ou o Custom GPT “SGAIM Design Critic”). É diferente do tutor socrático do MP1: em vez de guiar implementação, desafia decisões de design.
2. **Podem pedir ajuda de implementação** — “como uso o ChromaDB?” ou “como faço embed com Ollama?” são perguntas legítimas. A implementação não é o foco da avaliação.
3. **Não peçam à IA para desenhar o pipeline por vós nem para escrever a análise**. “Faz-me um RAG pipeline” ou “escreve as design decisions” produz texto que não conseguem defender na discussão.

A linha é a mesma do MP1: **usar IA para pensar mais, não para pensar menos**.

O que conta como plágio

- Copiar o pipeline de outro grupo
- Submeter análise escrita por IA sem a reformular e verificar contra o vosso trabalho real
- Apresentar comparações que não fizeram (inventar resultados)

Se as explicações na discussão não corresponderem ao documento, a nota reflecte a discussão.

Perguntas frequentes

P: Quanto tempo é que isto demora? Adaptar a pipeline da aula 07 ao vosso corpus é relativamente rápido — 1 a 3 horas. O benchmark, as comparações e a análise escrita demoram muito mais. Planeiem em conformidade.

P: Posso usar a API da OpenAI / Anthropic em vez de Ollama? Podem, mas o pipeline deve **também** funcionar com Ollama (modelo local). Comparar local vs API pode ser uma das vossas comparações.

P: Posso usar LangChain / LlamaIndex? Sim. Mas se usarem um framework, têm de saber explicar o que ele faz. “O LangChain trata disso” não é explicação. Na discussão, o docente pode perguntar: “o que é que o LangChain faz no retrieval que não poderias fazer com 10 linhas de Python?”

P: As 10 perguntas do benchmark — tenho de inventá-las eu? Sim. Devem ser perguntas que façam sentido para o vosso corpus e para as quais saibam a resposta. Se não sabem a resposta, não podem avaliar o sistema.

P: E se o meu pipeline parece funcionar sempre? Não funciona. Testem com armadilhas, com perguntas ambíguas, com perguntas que exijam cruzar múltiplos documentos. Se funciona sempre, as perguntas são demasiado fáceis — façam perguntas mais difíceis.

P: Trabalho a tempo inteiro. Qual é o mínimo? Pipeline da aula 07 adaptado ao vosso corpus + relatório com as 6 secções (mesmo que algumas sejam breves). Uma análise honesta de um pipeline simples vale mais do que um pipeline complexo com análise superficial.

Submissão

Prazo	Semana 10 (data exacta no Moodle)
Formato	ZIP com código + relatorio.pdf, submetido pelo Moodle
Discussão	Na aula, semana 10 — horário no Moodle

Submissões atrasadas perdem 20% por dia. Após 3 dias, não são aceites submissões.